

Urban Bigdata and AI Institute of University of Seoul

슈퍼컴퓨터 사용법 교육

2026.11.12.

01 교육프로그램 및 UBAI 소개

교육 목차

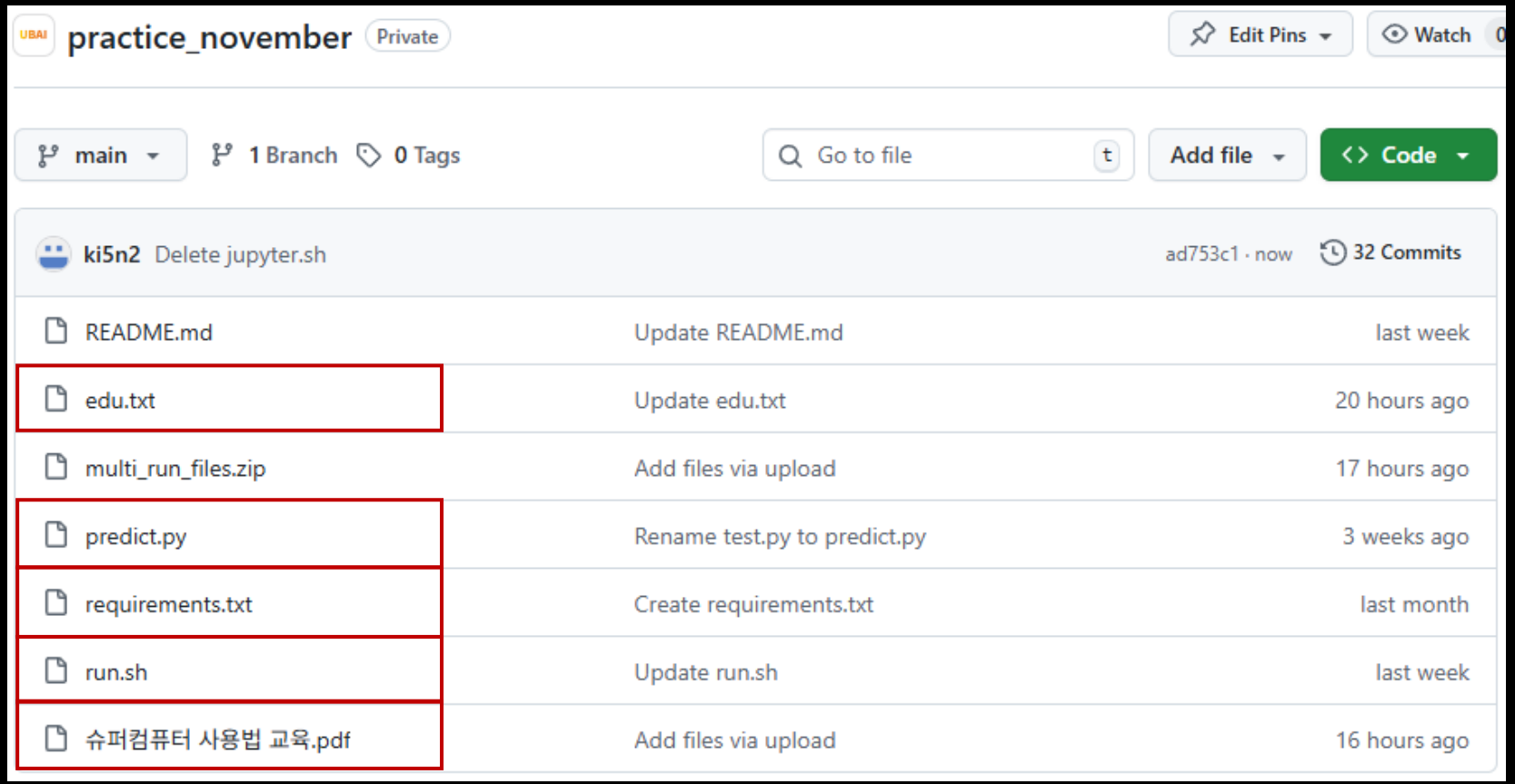
시작 시간	종료 시간	소요 시간	교육 내용
17:30	17:40	10min	교육 프로그램 및 UBAI 소개
17:40	18:10	30min	UBAI 서버 접속
18:10	18:30	20min	Python 환경 구축 (Miniconda)
18:30	18:40	10min	쉬는 시간
18:40	19:30	50min	슈퍼컴퓨팅 환경 이해 및 Job 제출

- WIFI: UBAI_CO_SPACE_01_5G/6G 또는 UBAI_CO_SPACE_02_5G/6G 로 접속
  - *PW: bigdata2025*

01 교육프로그램 및 UBAI 소개

교육 자료 다운로드

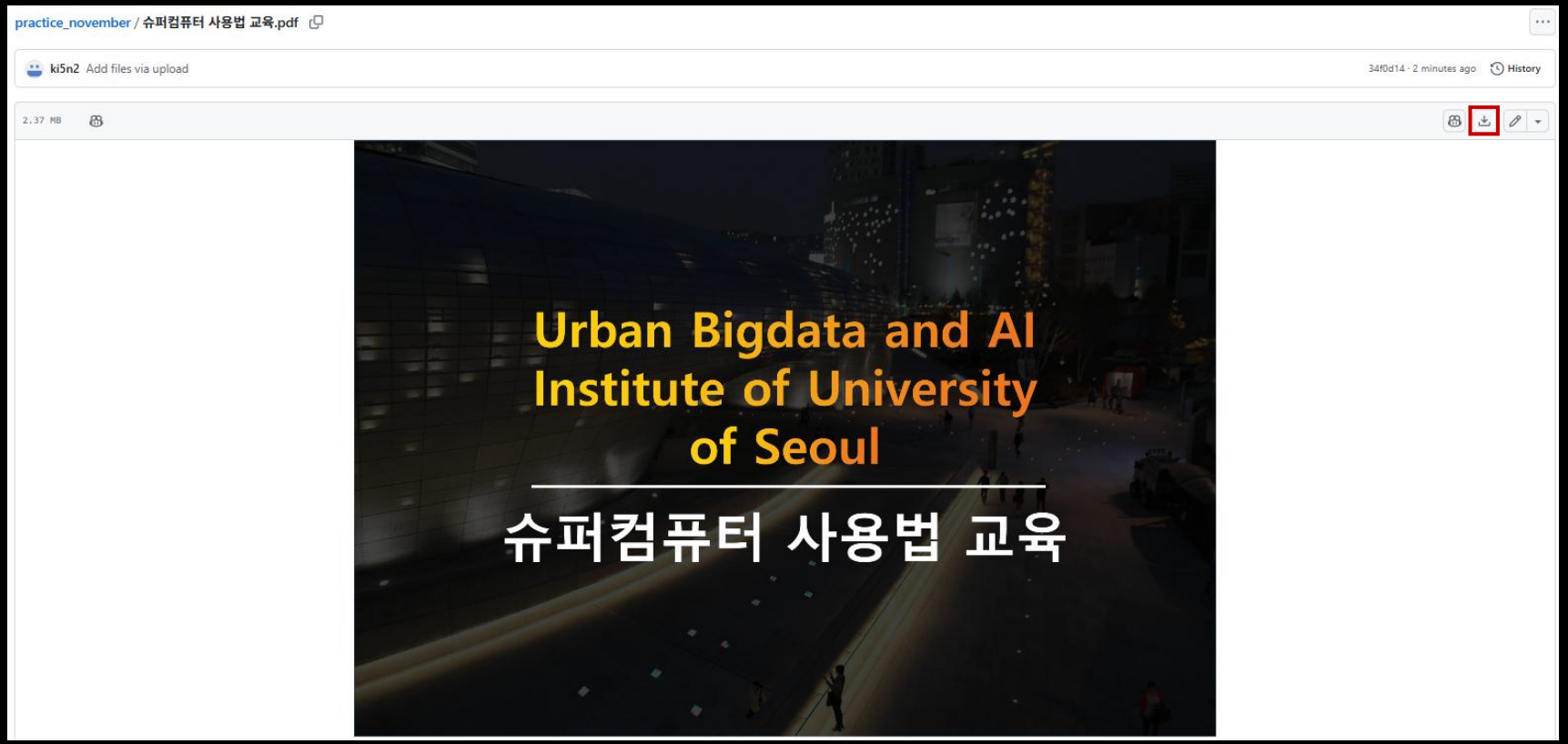
- https://github.com/uos-ubai/practice_november
- 슈퍼컴퓨터 사용법 교육.pdf, edu.txt, predict.py, requirements.txt, run.sh를 다운 받아주세요.



01 교육프로그램 및 UBAI 소개

교육 자료 다운로드

- https://github.com/uos-ubai/practice_november
- 슈퍼컴퓨터 사용법 교육.pdf, edu.txt, predict.py, requirements.txt, run.sh를 다운 받아주세요.



슈퍼컴퓨터란 무엇인가?

- 슈퍼컴퓨터(Supercomputer)는 일반 컴퓨터보다 훨씬 빠른 속도로
  방대한 계산을 수행할 수 있는 고성능 컴퓨터(HPC, High Performance Computing)입니다.

- 일반 PC가 1개의 CPU로 일처리를 한다면,
  슈퍼컴퓨터는 수많은 CPU, 또는 GPU를 동시에 사용하여 병렬로 작업을 처리합니다.

- 주로 기후 및 분자 시뮬레이션, 유전체 분석, 인공지능 모델 학습 등과 같은
  대규모 과학 및 공학 문제 해결에 사용됩니다.

도시과학빅데이터·AI연구원(UBAI) 운영방향

01 도시과학 연구

도시 경쟁력을 갖춘 선도도시의 기반이 되는
도시과학 연구

02 인재양성

빅데이터와 인공지능 분야의
역량을 갖춘 인재 양성

03 시스템 구축

창업 및 신규 첨단산업 생태계 구축
최첨단 빅데이터·인공지능 기술 개발 및 인프라
제공

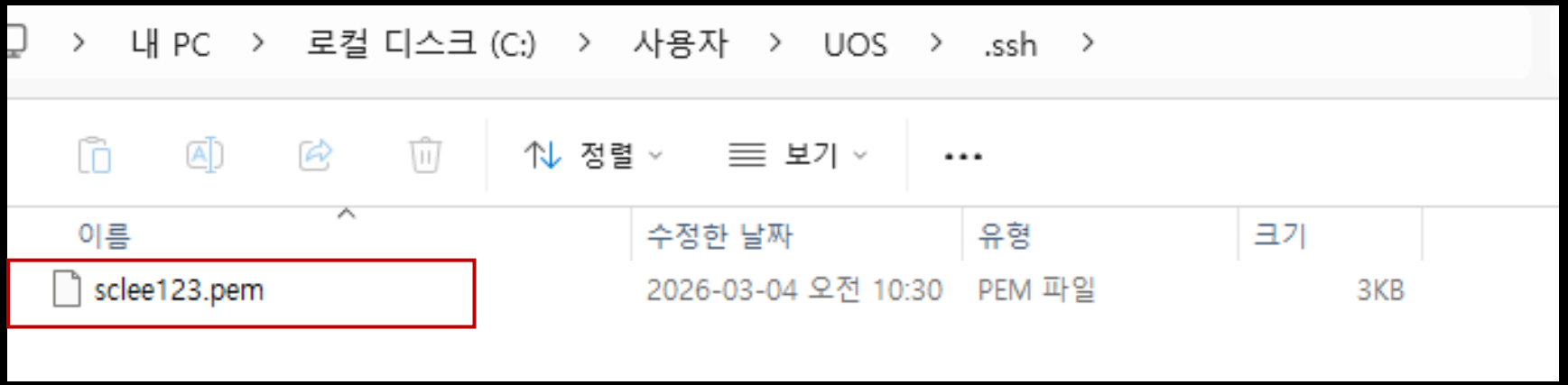
04 대·내외 협력

서울 및 국내외 기관과의 대내외 협력

02 UBAI 서버 접속

계정 생성

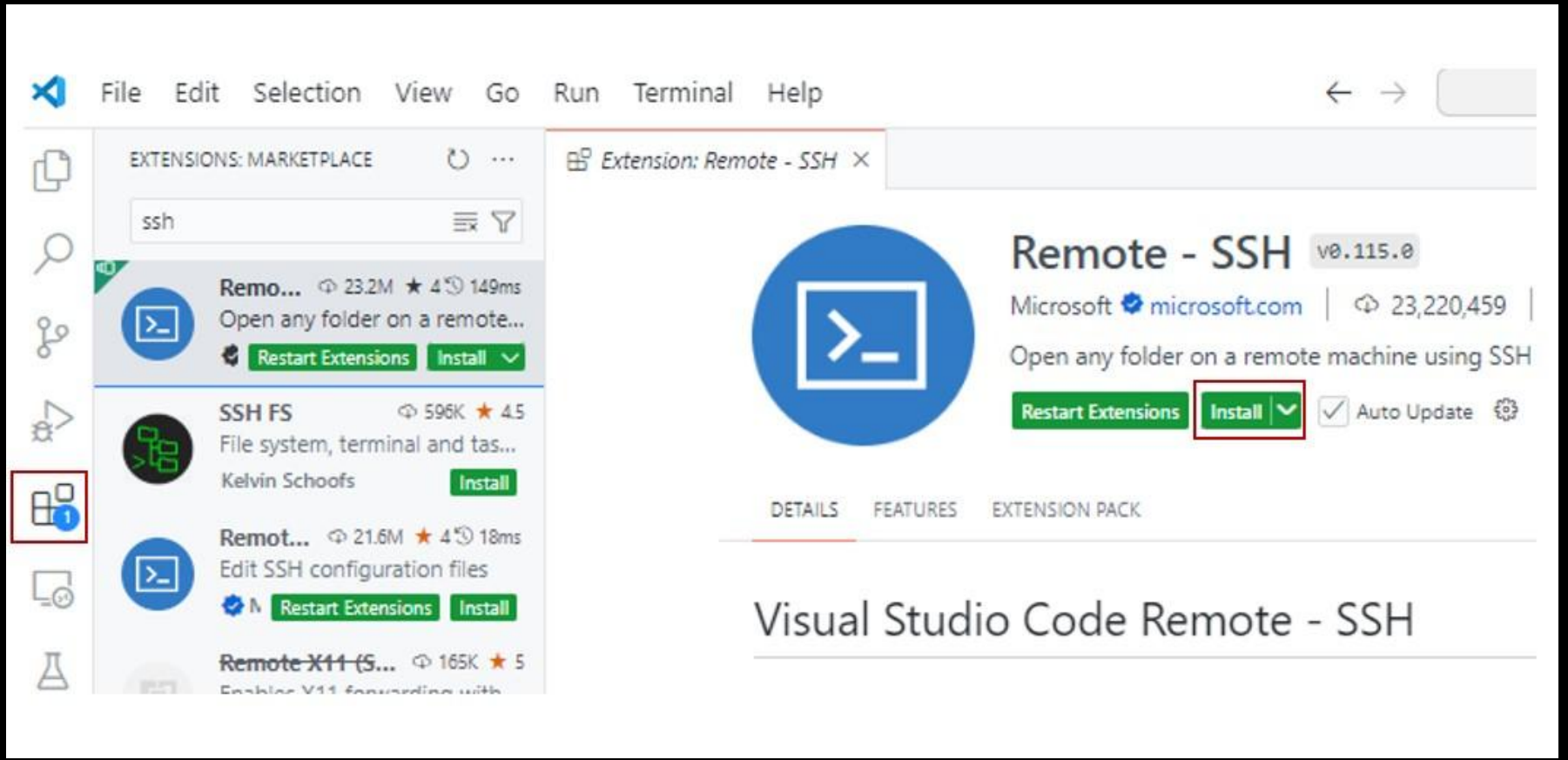
- UBAI Cluster 사용을 위해서는 사용자 계정을 발급받아야 합니다.
- 키 파일을 받은 상태에서 실습에 참가해야 합니다.
- 키 파일을 다음과 같은 경로에 저장합니다.(★ ★ ★)
  - Windows : C:₩Users₩사용자 이름₩.ssh₩사용자 계정.pem
  - MacOS : /Users/사용자 이름/.ssh/사용자 계정.pem
  - Mac에서는 Finder에서 Command + Shift + . 로 숨김 파일을 표시한 뒤 ~/.ssh 폴더에 들어가 .pem 파일을 저장하세요.



02 UBAI 서버 접속

접속 방법

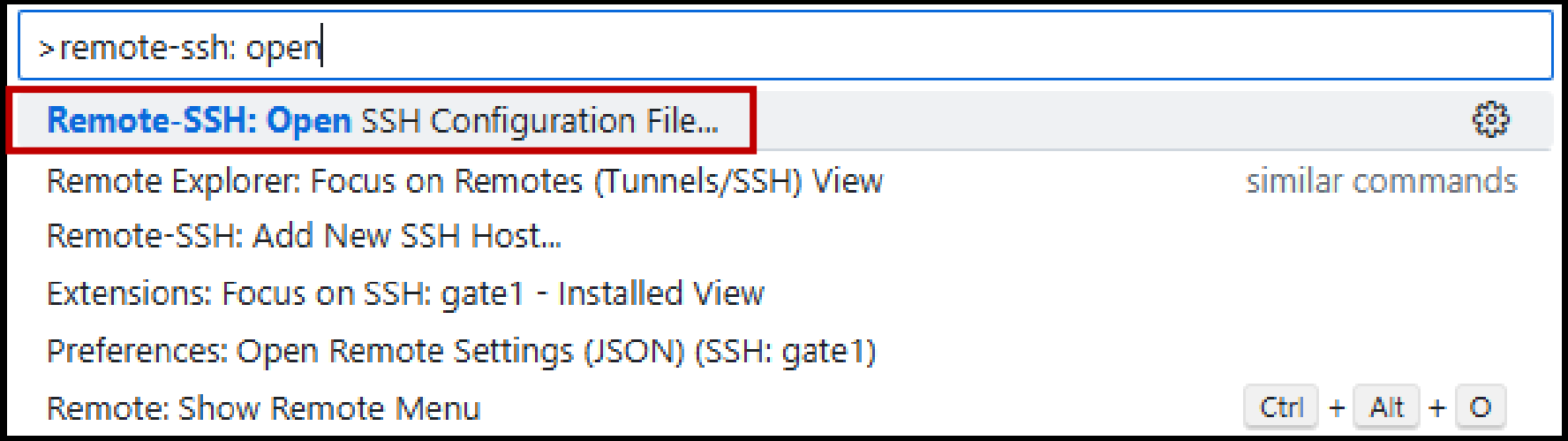
- Visual Studio Code 다운로드 및 접속
  - https://code.visualstudio.com/download
- Remote-SSH Extension 설치



02 UBAI 서버 접속

접속 방법

- Config 파일 수정
  - Ctrl + Shift + P
  - Remote-SSH: Open SSH Configuration File...

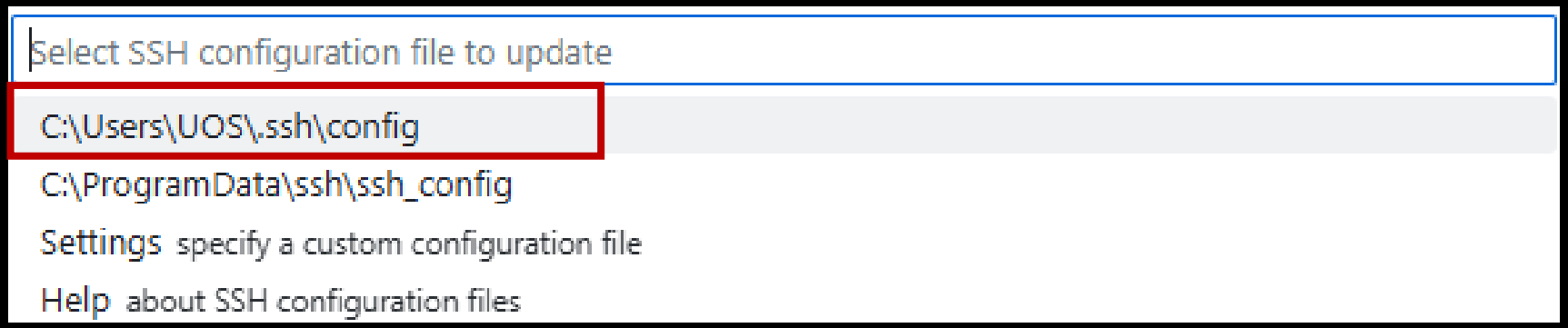


- .ssh폴더가 없는 경우에는 .ssh폴더와 config파일을 같이 생성
- .ssh폴더가 있는 경우에는 config파일만 생성

02 UBAI 서버 접속

접속 방법

- Config 파일 수정
  - 이후, 입력창에 C:₩Users₩사용자 이름₩.ssh₩config 선택



02 UBAI 서버 접속

접속 방법

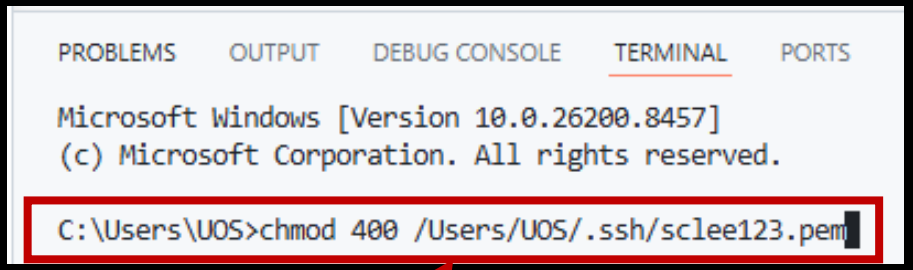
- Config 파일 수정
  - Host:
    - gate1
    - gate2
  - HostName:
    - 172.16.10.36(gate1인 경우)
    - 172.16.10.37(gate2인 경우)
  - Port: 22
  - User: 사용자 계정
  - IdentityFile: 앞서 저장한 키파일 경로
    - Windows : C:₩Users₩사용자 이름₩.ssh₩사용자 계정.pem
    - MacOS : /Users/사용자 이름/.ssh/사용자 계정.pem

config

C: > Users > UOS > .ssh > config

```
1   Host gate1
2       HostName 172.16.10.36
3       Port 22
4       User sclee123
5       IdentityFile C:\Users\UOS\.ssh\sclee123.pem
6
7   Host gate2
8       HostName 172.16.10.37
9       Port 22
10      User sclee123
11      IdentityFile C:\Users\UOS\.ssh\sclee123.pem
12
```

02 UBAI 서버 접속



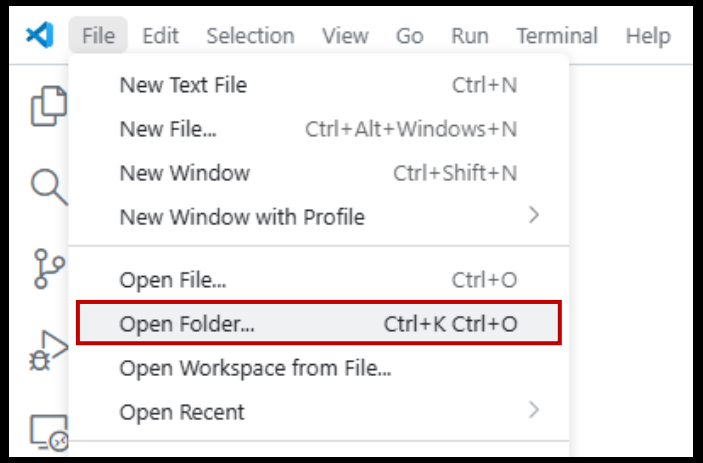
접속 방법(Only Mac User)

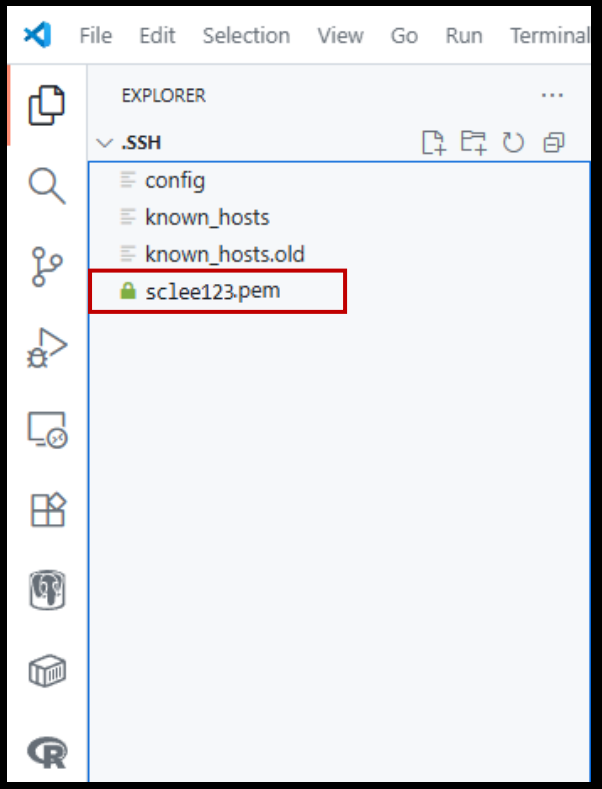
- MacOS 사용자는 다음과 같은 두 가지 추가 작업이 요구됨
- 키 권한 작업
  1. Terminal – New Terminal 열기(Ctrl + Shift + `)
  2. chmod 400 <비밀키 위치 경로 및 비밀키>
     ex) chmod 400 /Users/UOS/.ssh/sclee123.pem
        1. UOS를 사용자 이름으로 변경
        2. sclee123을 사용자 계정으로 변경
        3. SSH 접속용 키 파일 권한 설정 (최초 1회)
- 키 포맷 변경 작업
  1. VS code 내에서 키 파일(사용자 계정.pem) 열기
     - 키 파일 위치: /Users/사용자 이름/.ssh/사용자 계정.pem
  2. 하단 메뉴에서 파일 포맷 확인 후 클릭
  3. 파일 포맷 LF로 변경
  4. 변경 사항 저장(또는 덮어쓰기)

02 UBAI 서버 접속

접속 방법(Only Mac User)

- 파일 열기
  - File – Open Folder 선택
  - 키 파일 저장한 “/Users/사용자 이름/.ssh”폴더 열기

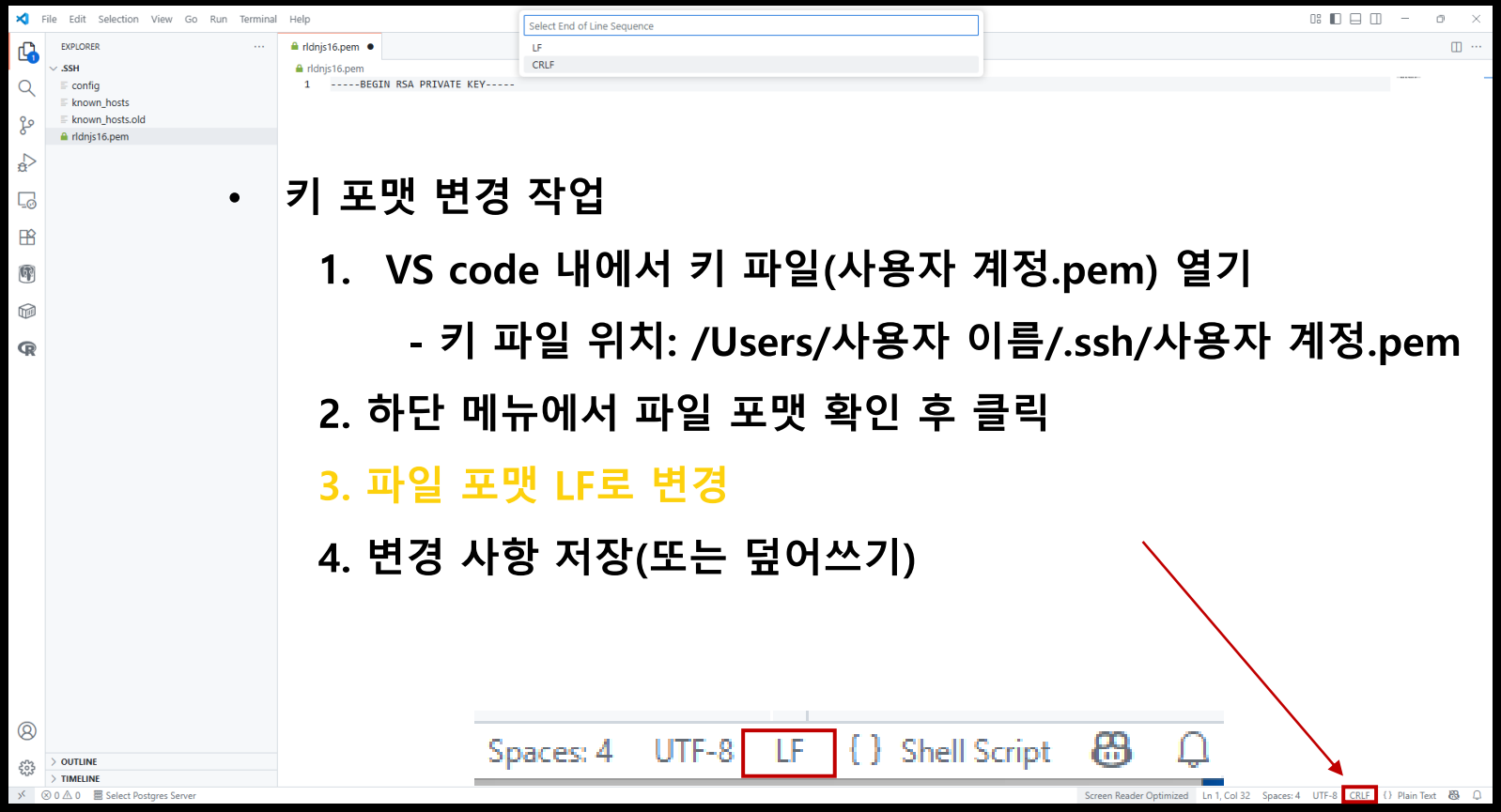




02 UBAI 서버 접속

접속 방법(Only Mac User)

- 파일 포맷 확인
  - 파일 포맷 확인 후 클릭

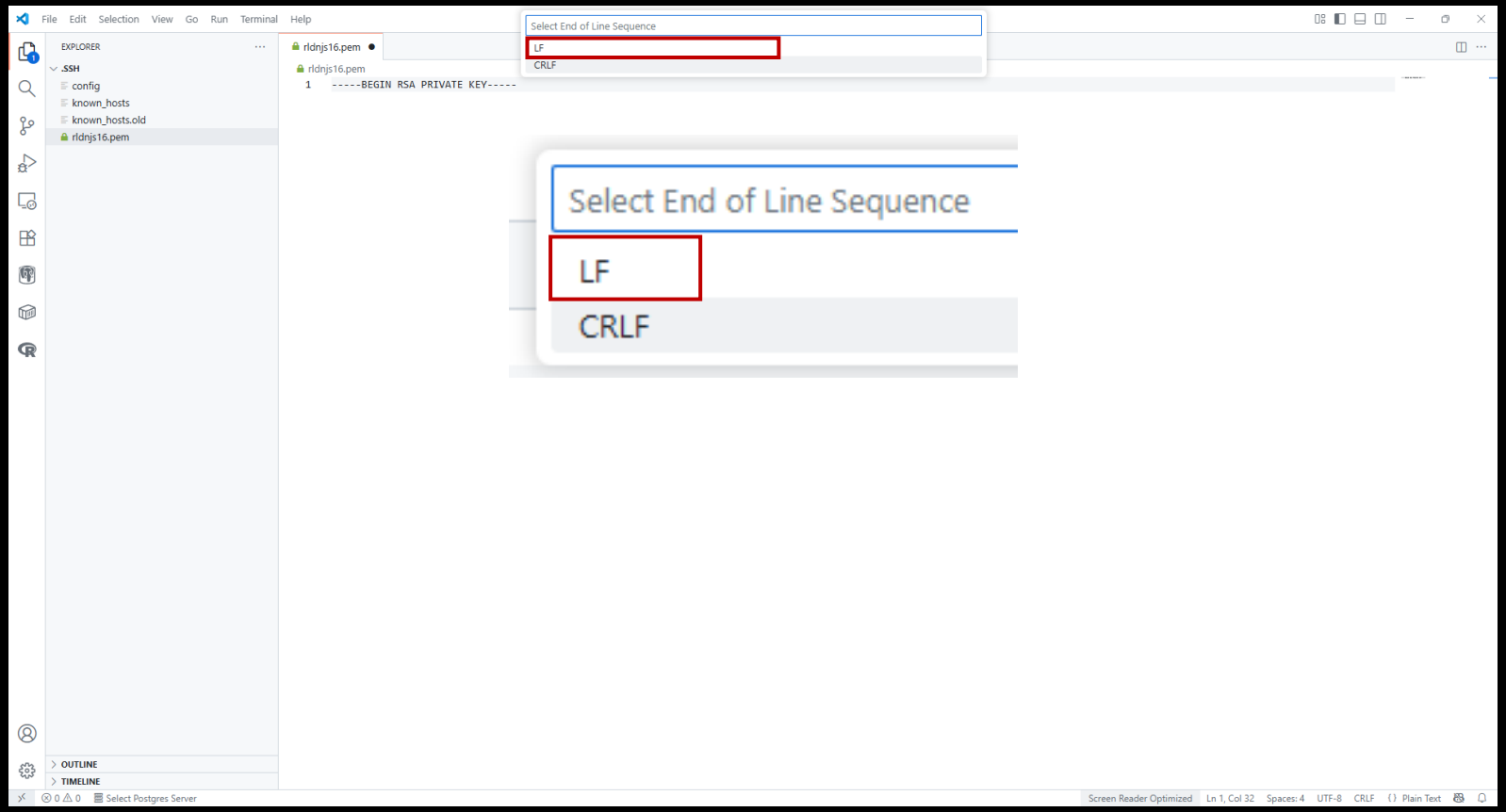


- 키 포맷 변경 작업
  1. VS code 내에서 키 파일(사용자 계정.pem) 열기
     - 키 파일 위치: /Users/사용자 이름/.ssh/사용자 계정.pem
  2. 하단 메뉴에서 파일 포맷 확인 후 클릭
  3. 파일 포맷 LF로 변경
  4. 변경 사항 저장(또는 덮어쓰기)

02 UBAI 서버 접속

접속 방법(Only Mac User)

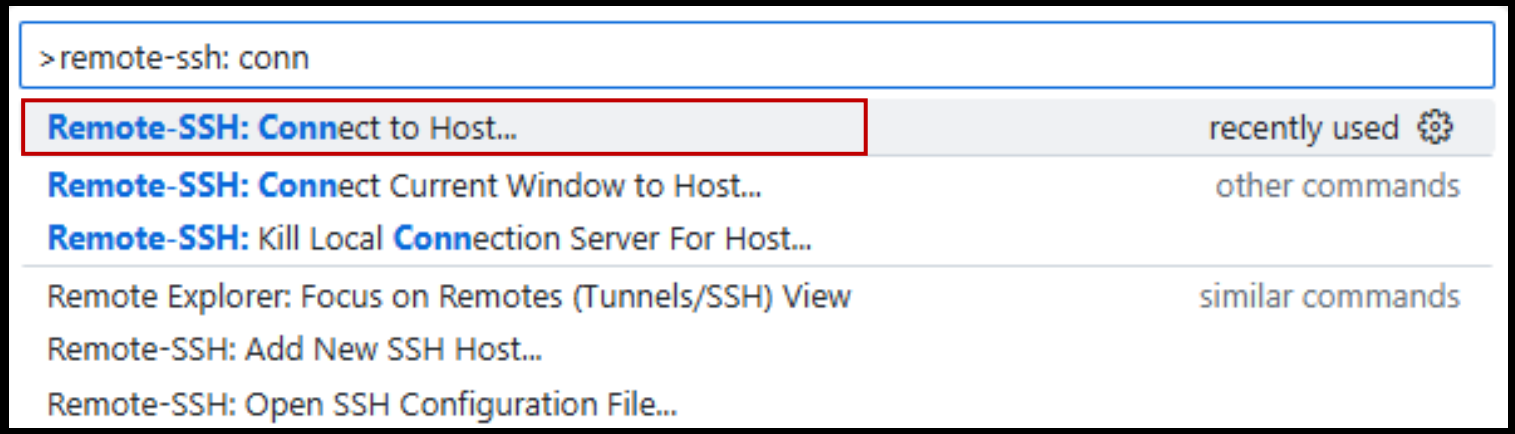
- 파일 포맷 변경
  - 파일 포맷 LF로 변경 후 저장



02 UBAI 서버 접속

접속 방법

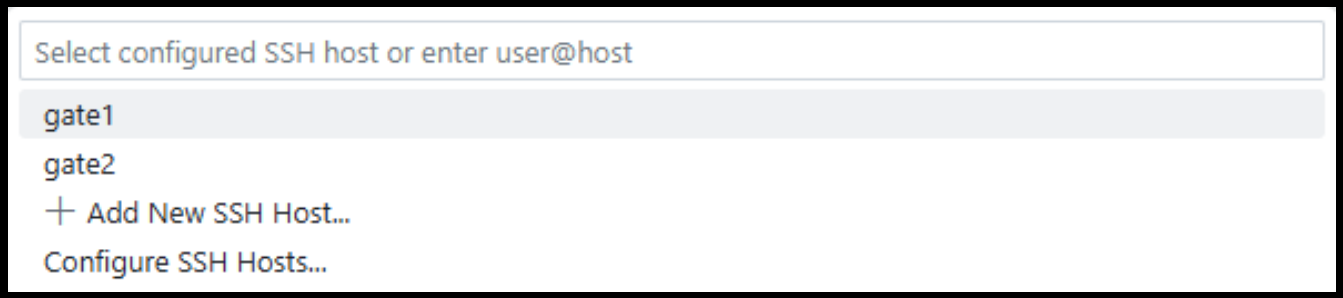
- SSH 접속
  - Ctrl + Shift + P
  - > Remote-SSH: Connect to Host...



02 UBAI 서버 접속

접속 방법

- SSH 접속
  - gate1 또는 gate2로 접속



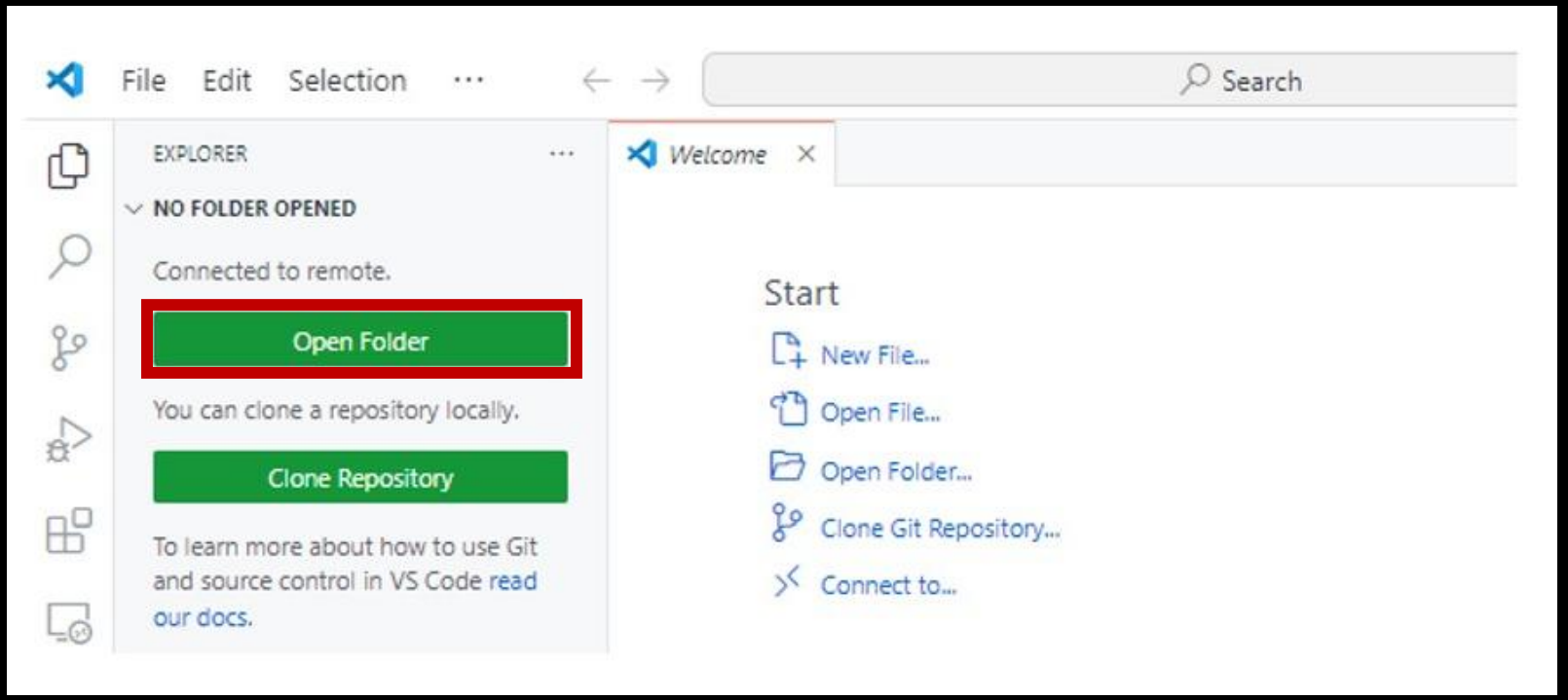
  - 운영체제 **반드시 Linux로 선택(★ ★ ★)**
  - **Continue** 선택
  - Gate 접속 확인 (왼쪽 하단)



02 UBAI 서버 접속

접속 방법

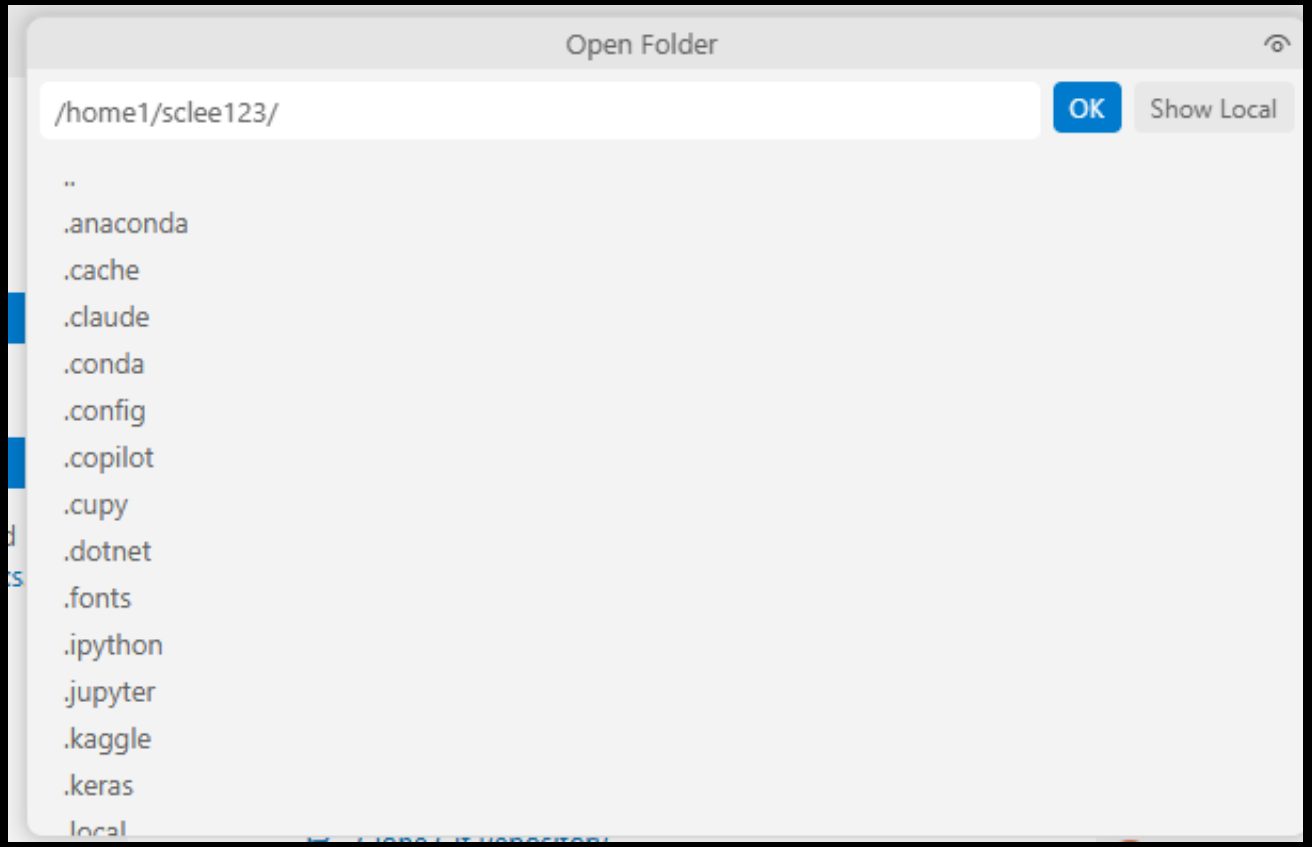
- 디렉토리 접속



02 UBAI 서버 접속

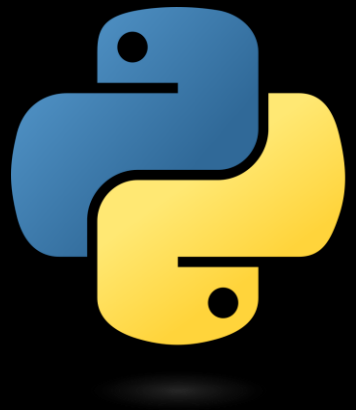
접속 방법

- 디렉토리 접속
  - 사용자(계정명) 폴더로 접속합니다.



03 Python 환경 구축

Python



- Python은 웹 어플리케이션, 소프트웨어 개발, 데이터 사이언스, 머신러닝, 딥러닝에 널리 사용되는 프로그래밍 언어입니다.
- 오픈소스 환경을 가지고 있으며, 많은 사람들이 이용하는 언어입니다.
- Python을 활용하기 위해서는 보통 Anaconda를 활용하지만 리눅스 컴퓨팅 환경에서는 Miniconda를 활용합니다.
- UBAI 슈퍼컴퓨터에서도 Python을 이용하기 위해서는 Miniconda를 사용해야 합니다.

03 Python 환경 구축

Miniconda

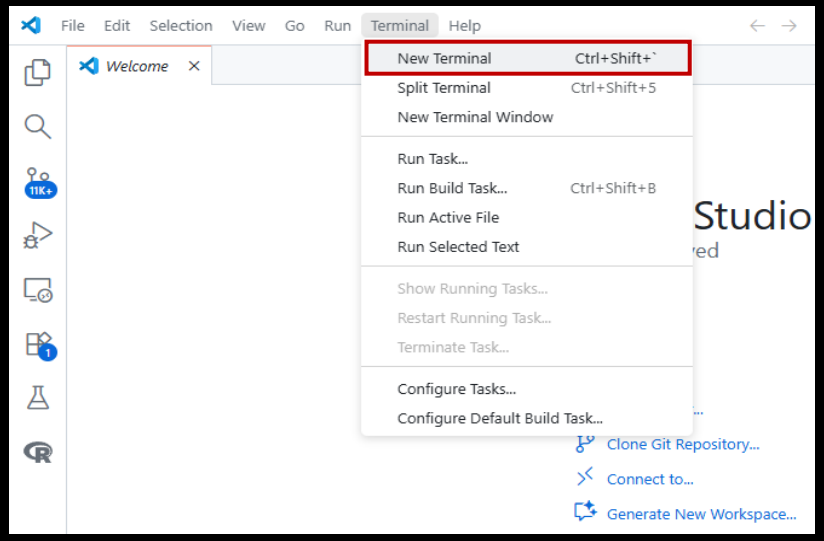
- Anaconda는 머신러닝이나 데이터 분석 등에 사용하는
  여러가지 패키지가 기본적으로 포함되어있는 파이썬 배포판입니다.
- Python의 가상환경을 구축하는데 매우 유용하게 사용됩니다.
- Miniconda는 Anaconda의 경량 버전이라고 볼 수 있습니다.



03 Python 환경 구축

Miniconda 설치

- Terminal 접속 (Ctrl + Shift + `)



03 Python 환경 구축

Miniconda 설치

- Miniconda 설치 파일 다운로드
  - 앞서 github에서 다운로드한 edu.txt에 저장된 명령어를 복사 후 붙여 넣어줍니다.
  - wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(base) [sclee123@gate1 ~] wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

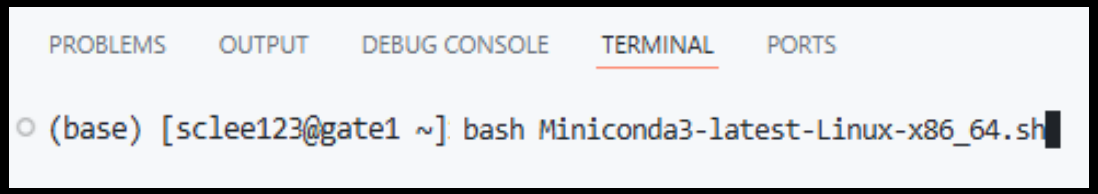
```
(base) [sclee123@gate1 ~]$ wget https://repo.anaconda.com/miniconda/Minico
nda3-latest-Linux-x86_64.sh
--2026-06-08 13:50:20--  https://repo.anaconda.com/miniconda/Miniconda3-la
test-Linux-x86_64.sh
Resolving repo.anaconda.com (repo.anaconda.com)... 104.16.32.241, 104.16.1
91.158, 2606:4700::6810:bf9e, ...
Connecting to repo.anaconda.com (repo.anaconda.com)|104.16.32.241|:443...
connected.
HTTP request sent, awaiting response... 200 OK
Length: 163179296 (156M) [application/octet-stream]
Saving to: ‘Miniconda3-latest-Linux-x86_64.sh.5’

Miniconda3-latest- 100%[==============>] 155.62M  24.5MB/s    in 8.4s

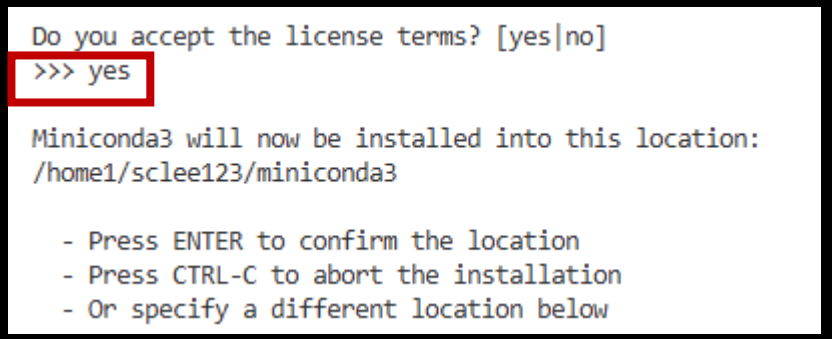
2026-06-08 13:50:28 (18.5 MB/s) - ‘Miniconda3-latest-Linux-x86_64.sh.5’ sa
ved [163179296/163179296]
```

Miniconda 설치

- Bash 명령어를 통해 Miniconda 설치
  - bash Miniconda3-latest-Linux-x86_64.sh
  - 라이센스 동의에 대한 내용(Do you accept the license terms? [yes|no])이 나타날 때까지 Enter를 여러 차례 눌러줍니다.



  - 그 후 라이센스 동의 확인에 대한 질문에 yes라고 입력해줍니다.



03 Python 환경 구축

Miniconda 설치

- Bash 명령어를 통해 Miniconda 설치

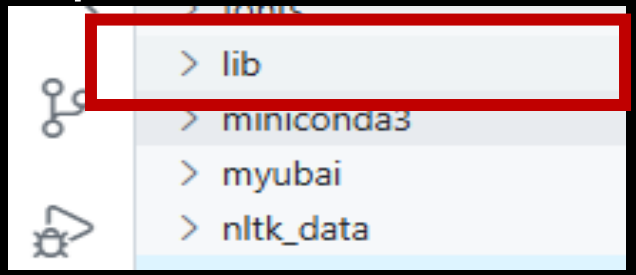
  1. 화면에서 해당 경로가 자신의 서버에 있는 경로와 맞는지 확인 후 Enter를 눌러줍니다.
  (Miniconda3 will now be installed into this location:
  /home1/<리눅스 계정명>/miniconda3)

  2. 설치가 완료되면, conda init 선택에 대한 질문이 나타납니다.
  (Do you wish to update your shell profile to automatically initialize conda?)
  Yes를 입력하신 후 enter를 눌러 주시면 됩니다.

  3. 변경사항 반영을 위해 지금 작업한 터미널을 닫고 새로운 터미널로 접속해주세요.

  4. 재접속 시, 터미널 프롬프트에 (base)[사용자ID@gate1/2]가 보인다면
  성공적으로 설치가 완료된 것입니다.

  5. 설치 후 왼쪽의 탐색기(Explorer) 목록에 miniconda3 폴더가 있는지 꼭 확인해주세요.



03 Python 환경 구축

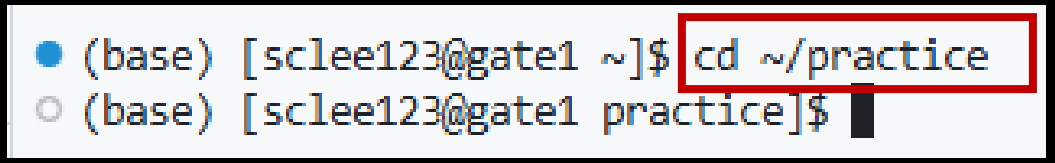
Miniconda 실행

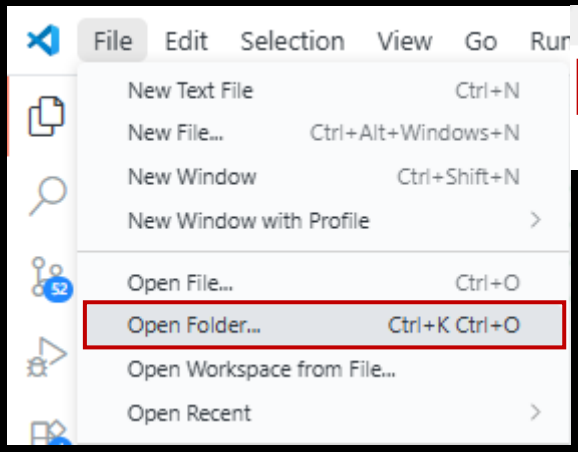
- 사용자 계정 내 임의의 폴더를 생성합니다
  - ex) practice

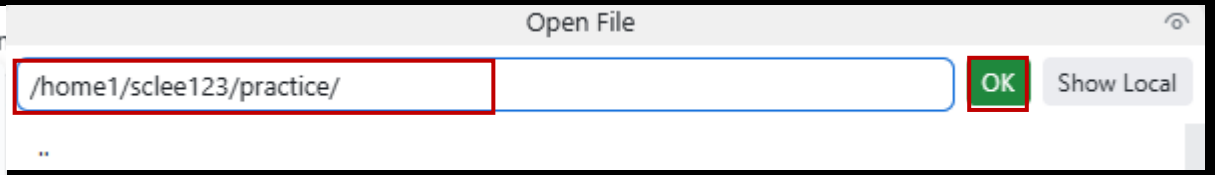
  mkdir ~/practice



- 생성한 폴더로 이동합니다.
  - /home1/사용자 계정/폴더 이름/
  - **cd ~/practice**



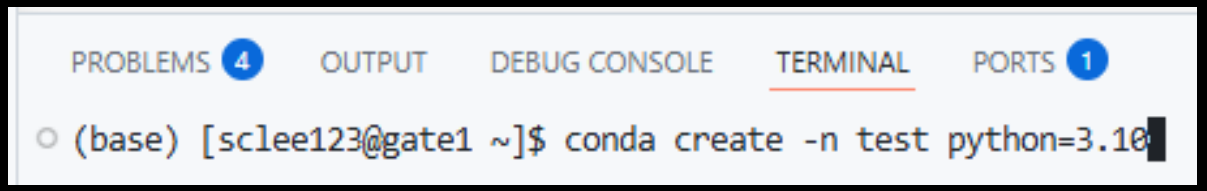




03 Python 환경 구축

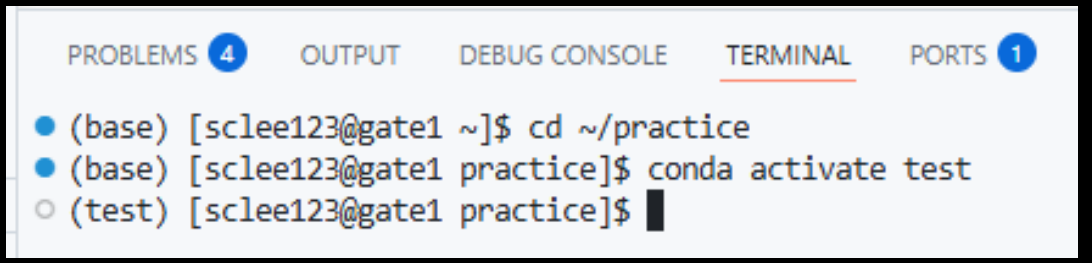
Miniconda 실행

- 가상 환경 생성
  - conda create –n {가상환경_이름} python={설치할_python_version}
  - Ex) 가상환경 이름을 test로 설정하겠습니다.



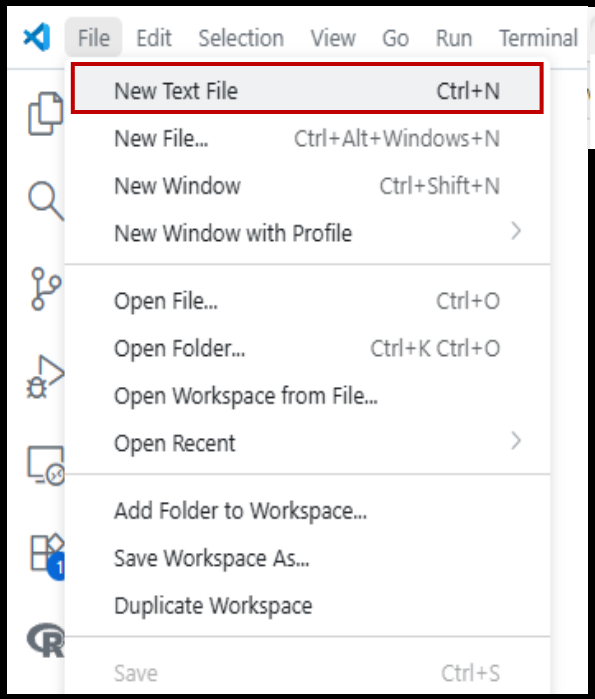
  - “Do you accept the Terms of Service”가 나타나는 경우, a를 입력하신 후 enter를 눌러 주세요.
  - Proceed ([y]/n)? y 입력

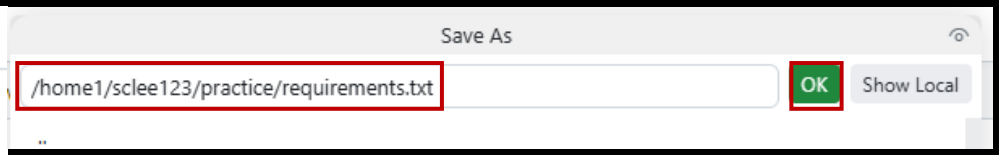
- 가상 환경 실행
  - conda activate {가상환경 이름}
  - ex: conda activate test



Miniconda 실행

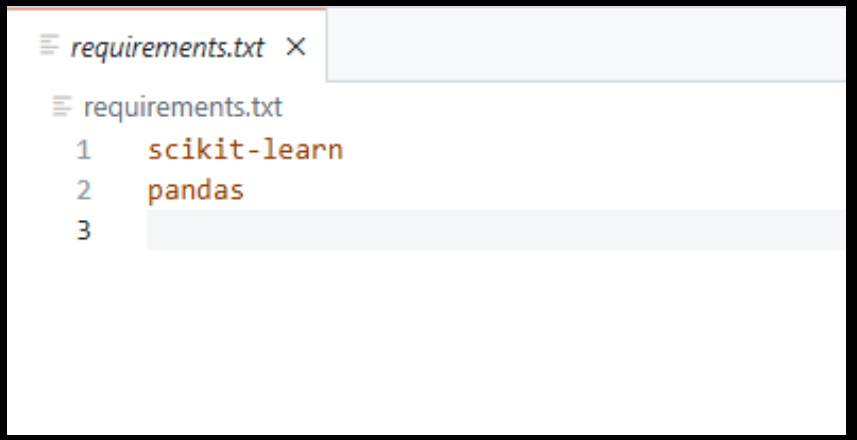
- requirements.txt
  - 앞서 github에서 다운로드한 requirements.txt의 내용을 복사(또는 다운로드)한 후, 새 파일에 붙여넣고 requirements.txt라는 이름으로 저장(Ctrl + s)해 주세요.

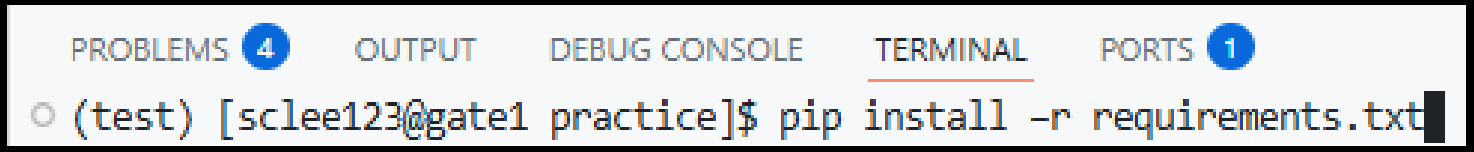




Miniconda 실행

- requirements.txt
  - terminal에서 가상환경 접속 후 pip install –r requirements.txt를 입력합니다.
  - 간단한 실습을 위해, 필요한 패키지들을 한 번에 설치합니다.
  - 이후 가상환경에서 별도 설치 없이 패키지를 사용할 수 있습니다.






Break Time

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

Slurm

- 도시과학빅데이터·AI연구원은 사용자에게 고성능컴퓨팅(HPC) 자원을 제공합니다.
- 사용자는 Slurm을 통해 독점적인 자원을 할당받고
  AI학습 및 추론, 연산, 시뮬레이션 등의 작업(Job)을 수행할 수 있습니다.

- Slurm은 다양한 사용자들의 다양한 요구를 수용하고(Job Submit),
  각 사용자들의 작업을 스케줄링하며(Task Scheduling),
  자원을 관리(Resource Management)하는 Linux 유틸리티 입니다.
- 여러 명의 사용자가 UBAI 클러스터를 이용하는데 있어서
  원활한 실험 및 계산을 위한 중재자 역할을 수행합니다.

- UBAI는 여러 명의 사용자에게 UBAI Cluster를 제공하기 위해 Slurm을 사용합니다.

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

Cluster

- 여러 대의 컴퓨터들이 연결되어 하나의 시스템처럼 동작하는 컴퓨터들의 집합을 말합니다.

Partition

- Partition은 특정 자원 그룹을 정의하는 논리적 단위입니다.
- 사용자들이 작업을 제출할 때 특정 Partition을 지정하여 자원을 할당받을 수 있습니다.

Partition	# of Nodes	# of Cores/ node	CPU	GPU/node	Memory/ node
gpu1	14(n001-n014)	48	Intel Xeon Gold 6240R	**RTX3090(4EA)**	768GB
gpu6	25(n015-n039)	48	Intel Xeon Gold 6240R	**A10(4EA)**	768GB
cpu1	10(n040-n049)	48	Intel Xeon Gold 6240R	-	768GB
gpu2	11(n051-n061)	56	Intel Xeon Gold 6348R	**A10(4EA)**	1024GB
gpu3	10(n062-n071)	56	Intel Xeon Gold 6348R	**A6000ada(4EA)**	1024GB
gpu4	29(n072-n100)	56	Intel Xeon Gold 6348R	**A6000(4EA)**	1024GB
gpu5	6(n101-n106)	64	Intel Xeon Platinum-8358	**A6000(4EA)**	1024GB
cpu2	10(n107-n116)	256	AMD EPYC 9755 128-Core Processor	-	1024GB

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

Node

- 클러스터를 구성하는 개별 컴퓨터(서버)를 의미합니다.
- 각 Node는 CPU, 메모리, GPU, 디스크 등의 자원을 갖습니다.

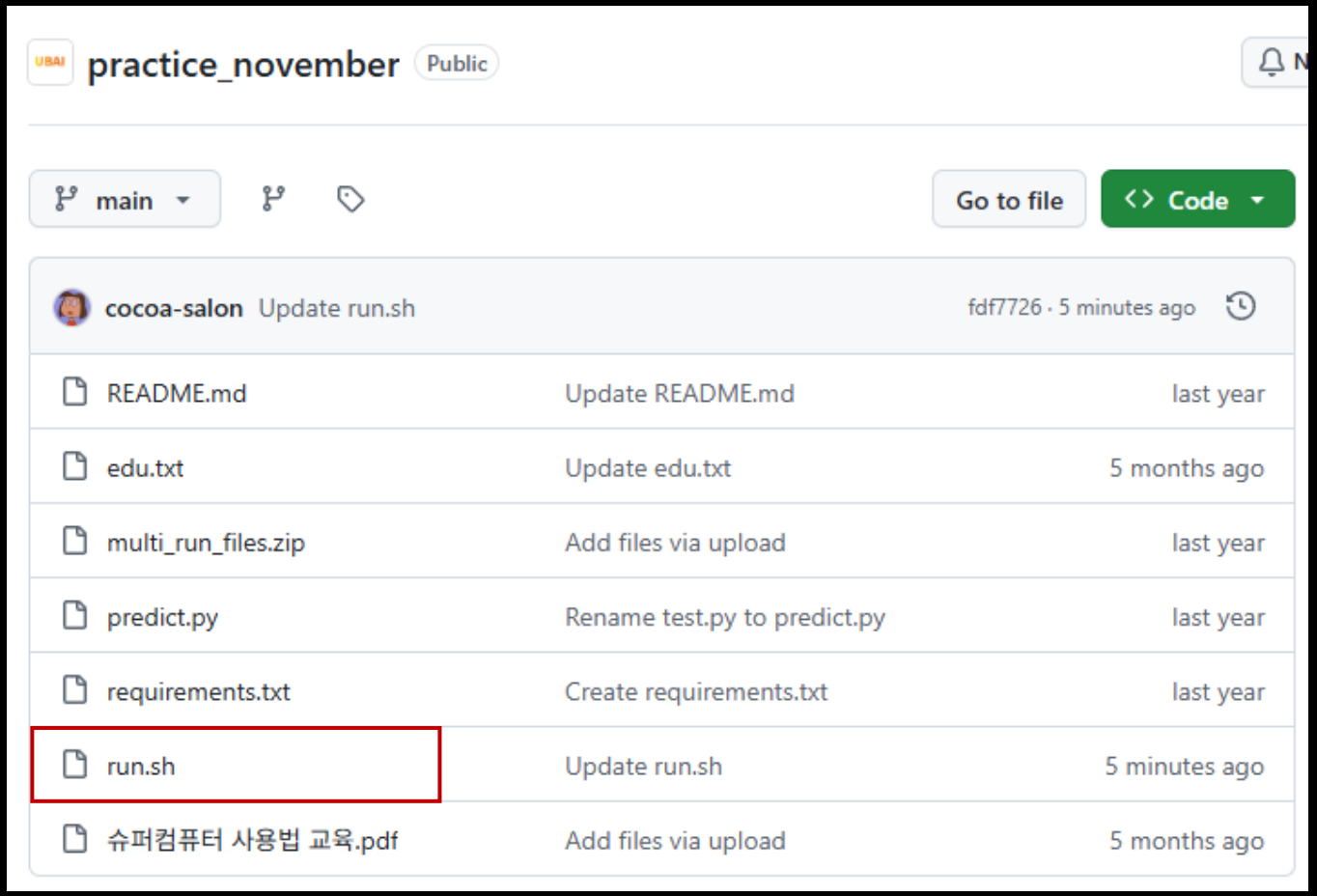
Job

- 컴퓨터 클러스터와 같은 경우
  스케줄러(이하 Slurm)를 사용하여 컴퓨터 자원에 접근하기 위한 요청을 합니다.
- 이를 작업(Job)이라고 부르며,
  이는 유저가 원하는 프로그램이나 스크립트를 실행하고 싶을 때의 행동을 지칭하기도 합니다.

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

배치 파일 생성

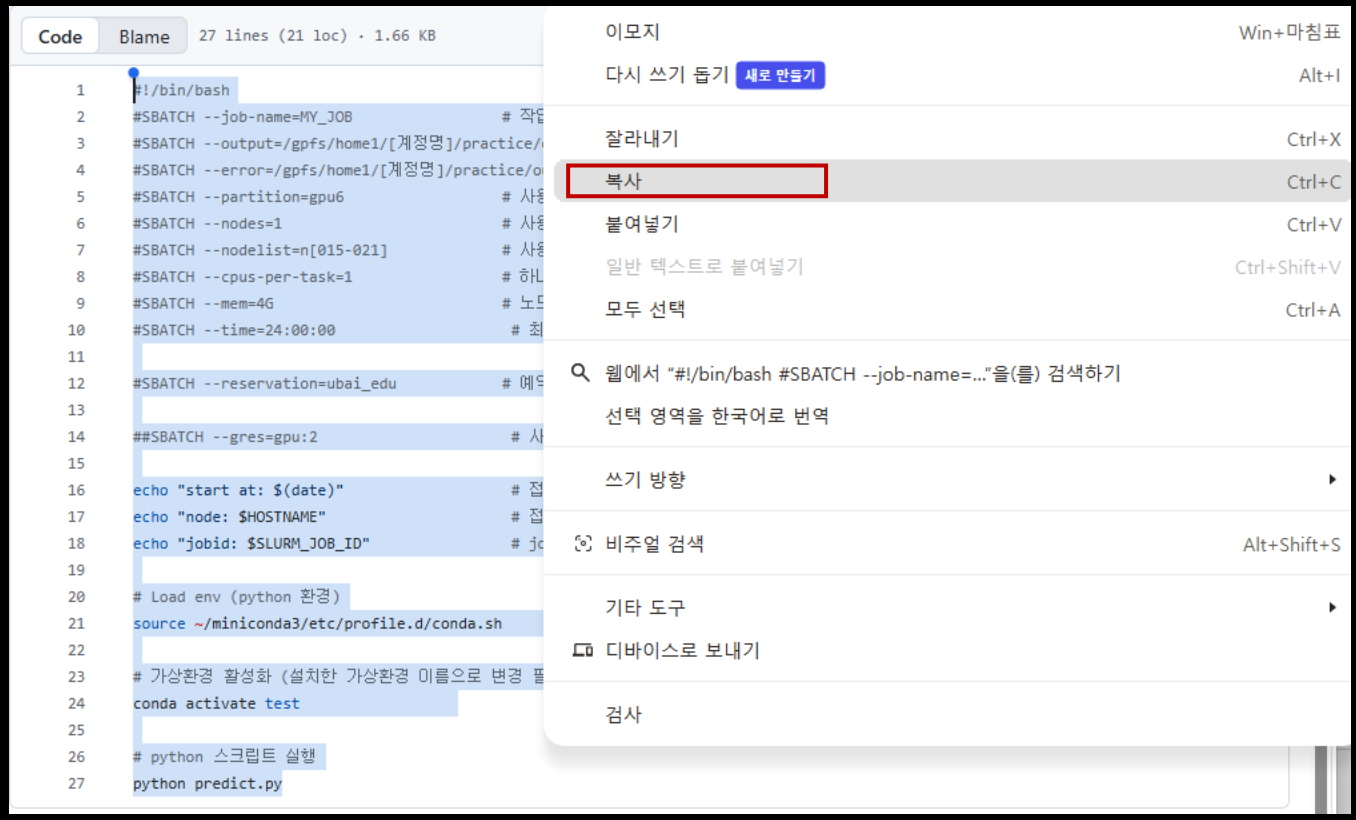
- batch file: 반복적인 명령어를 자동화하기 위해 사용하는 텍스트 형식의 스크립트 파일



04 슈퍼컴퓨팅 환경 이해 및 Job 제출

배치 파일 생성

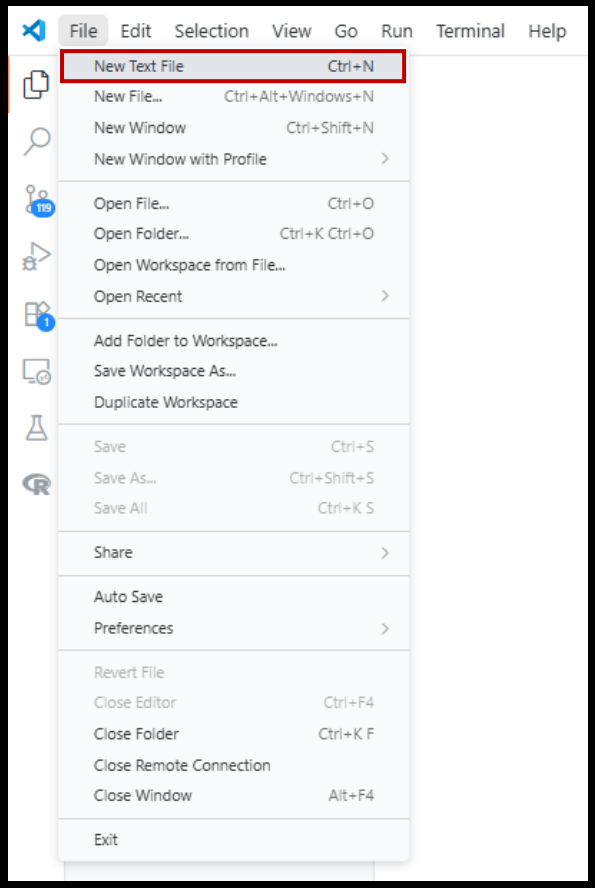
- 배치 파일을 만들어주기 위해, 파일을 전부 복사(또는 다운로드)해줍니다.



04 슈퍼컴퓨팅 환경 이해 및 Job 제출

배치 파일 생성

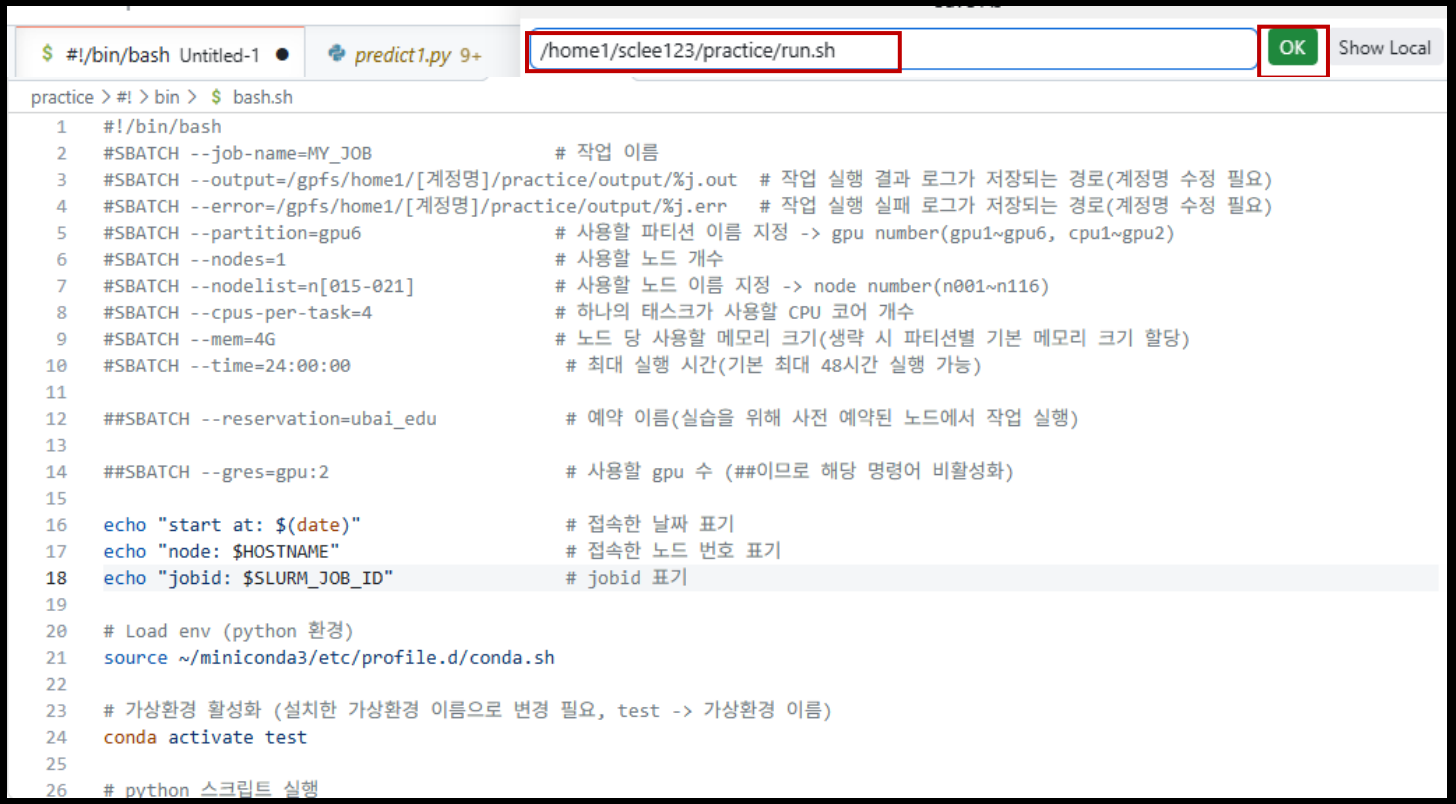
- 배치 파일 생성을 위해 새로운 텍스트 파일을 만들어줍니다.



04 슈퍼컴퓨팅 환경 이해 및 Job 제출

배치 파일 생성

- *파일명*.sh로 저장해줍니다.(예시, run.sh)



04 슈퍼컴퓨팅 환경 이해 및 Job 제출

- 프로젝트 수행을 위한 배치 파일을 생성합니다.

- 다음의 사항을 본인의 작업(job)에 맞게 입력한 후, 본인이 원하는 파일의 이름을 지정하여 filename.sh 형식으로 파일을 저장합니다.
- 예를 들어, run.sh로 저장해보겠습니다.

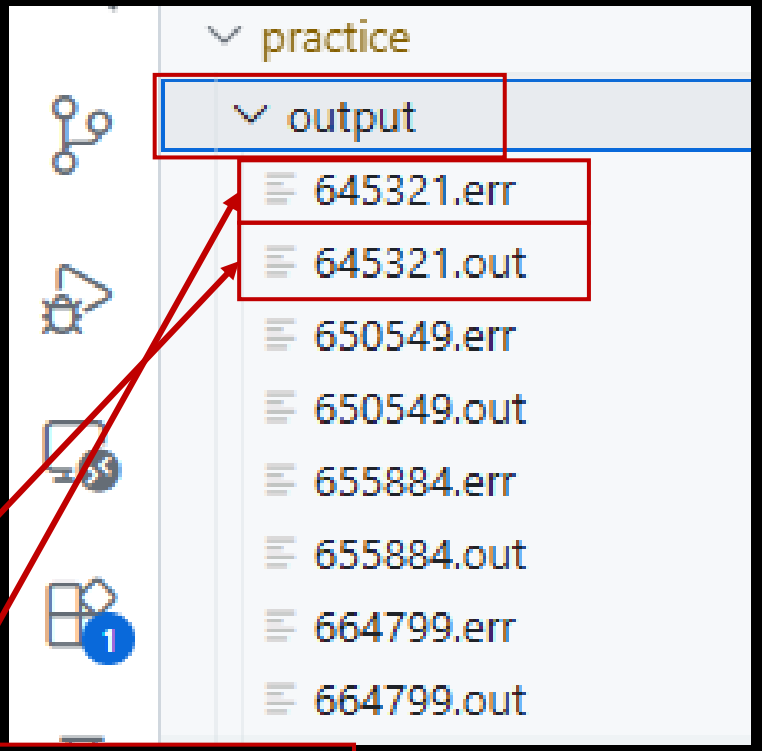
- 파일명이 .sh 형식인지 반드시 확인해주세요.

practice > #! > bin > $ bash.sh

```
1   #!/bin/bash
2   #SBATCH --job-name=MY_JOB                    # 작업 이름
3   #SBATCH --output=/gpfs/home1/[계정명]/practice/output/%j.out
4   #SBATCH --error=/gpfs/home1/[계정명]/practice/output/%j.err
5   #SBATCH --partition=gpu6                     # 사용할 파티션 이름
6   #SBATCH --nodes=1                            # 사용할 노드 개수
7   #SBATCH --nodelist=n[015-021]                # 사용할 노드 이름 지
8   #SBATCH --cpus-per-task=4                    # 하나의 태스크가 사용
9   #SBATCH --mem=4G                             # 노드 당 사용할 메모
10  #SBATCH --time=24:00:00                       # 최대 실행 시간(기본
```

```
#SBATCH --partition=gpu6
#SBATCH --nodes=1
#SBATCH --nodelist=n[015-021]
#SBATCH --cpus-per-task=4
#SBATCH --mem=4G
#SBATCH --time=24:00:00
```

04 슈퍼컴퓨팅 환경 이해 및 Job 제출



배치 파일 생성

#SBATCH --job-name=JOB_NAME

- 작업 이름을 지정하는 명령어입니다.
- Ex) #SBATCH --job-name=MY_JOB

#SBATCH –output=./output/%j.out

- 작업 결과 파일의 저장 위치를 지정하는 명령어입니다.
- Ex) #SBATCH --output=/gpfs/home1/[계정명]/practice/output/%j.out

#SBATCH –error=./output/%j.err

- 작업 중 발생하는 오류 메시지의 파일 저장 위치를 지정하는 명령어입니다.
- Ex) #SBATCH --error=/gpfs/home1/[계정명]/practice/output/%j.err

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

```
#SBATCH --partition=gpu6
#SBATCH --nodes=1
#SBATCH --nodelist=n[015-021]
#SBATCH --cpus-per-task=4
#SBATCH --mem=4G
#SBATCH --time=24:00:00
```

배치 파일 생성

#SBATCH --partition=gpu_n

- 사용할 Partition을 지정하는 명령어입니다.
- 지정된 노드가 속한 파티션을 입력해줍니다.
- 사용할 노드 대응되는 Partition 번호로 변경해줍니다. (gpu1 ~ gpu6)

#SBATCH –nodelist=n032,n033

- 사용할 노드를 지정하는 명령어입니다.
- 생략 시 해당 파티션 내 자동 배정되므로 생략해도 무방합니다.
  - (원활한 실습 진행을 위해 특정 노드 지정)
- 사용하고자 하는 노드 번호로 변경해줍니다. (n001 ~ n106)
- 지정한 노드 번호에 맞추어 Partition 번호도 변경해줍니다.
- 노드 번호와 Partition 번호가 올바르게 대응되지 않은 경우, 다음과 같은 에러가 발생합니다.

```
(test) [rldnjs16@gate1 practice]$ sbatch run.sh
sbatch: error: Batch job submission failed: Invalid node name specified
```

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

배치 파일 생성

#SBATCH --gres=gpu:1

- 몇 개의 GPU를 사용할 것인지 지정하는 명령어입니다.
- 각 노드는 4개의 GPU를 가지고 있으며, 해당 옵션을 사용하면 해당 노드의 GPU 중 1개를 현재 작업에 할당함을 의미합니다.

#SBATCH --cpus-per-task=1

- 총 필요한 CPU 코어의 개수를 지정하는 명령어입니다.
- 각 파티션 마다 CPU 코어의 개수가 다릅니다.
- 사용자의 작업에 따라 요구하는 CPU 자원이 다르므로, 작업에 맞게 코어 개수를 설정합니다.

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

배치 파일 생성

#SBATCH --mem=4G

- 작업을 실행할 때 필요한 메모리를 지정합니다.
- 메모리 옵션을 지정하지 않는 경우 노드 당 기본 메모리(4G부터)가 할당됩니다(파티션별 상이).
- 하나의 노드에는 다수의 이용자가 제출한 작업이 실행중인 경우가 일반적입니다. 하나의 작업이 노드의 메모리를 과도하게 점유하지 않도록 메모리 지정 시 유의해 주시기 바랍니다(노드의 가용 메모리가 부족해져 타 연구자의 작업이 중단될 수 있습니다.)

#SBATCH --time=48:00:00

- 작업에 필요한 실행시간을 지정 할 수 있습니다(생략시 기본 최대 48시간이 적용됨 “**##SBATCH --time=48:00:00**”).
- 실행 시간을 과도하게 길게 지정할 경우 제출된 작업이 실행되기까지 상당시간의 지연이 발생될 수 있습니다.

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

배치 파일 생성

echo "start at: $(date)”

- 접속 날짜가 표기됩니다.

echo "node: $HOSTNAME"

- 접속한 노드 번호가 표기됩니다.

echo "jobid: $SLURM_JOB_ID"

- jobid가 표기됩니다.

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

배치 파일 생성

module ~

- 원하는 Linux 환경을 구축할 수 있고, 기본적으로 CUDA/11.2.2 실행으로 셋팅되어 있습니다.
- 지금과 같이 다른 GPU 환경을 원할 경우, 원하는 모듈을 load 합니다.
- GPU 환경을 사용하고 싶은 경우에만 해당하며, GPU 환경을 사용하지 않을 경우(CPU Partition 사용) 지우셔도 무관합니다.

source ~/miniconda3/etc/profile.d/conda.sh

- conda 환경을 사용하기 위한 초기 설정을 불러오는 작업입니다.
- 해당 명령을 통해 conda activate 등 conda 관련 명령어들을 사용할 수 있습니다.
- Slurm 같은 비인터랙티브 셸은 보통 ~/.bashrc를 읽지 않아 conda가 인식되지 않으므로, 스크립트(.sh 파일)에 source ~/miniconda3/etc/profile.d/conda.sh로 conda 함수를 로드해야 합니다.

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

배치 파일 생성

conda activate test

- 본인이 생성한 가상환경을 활성화시켜,
  해당 환경에서 설치된 패키지들을 사용하여 작업을 진행할 수 있습니다.

python predict.py

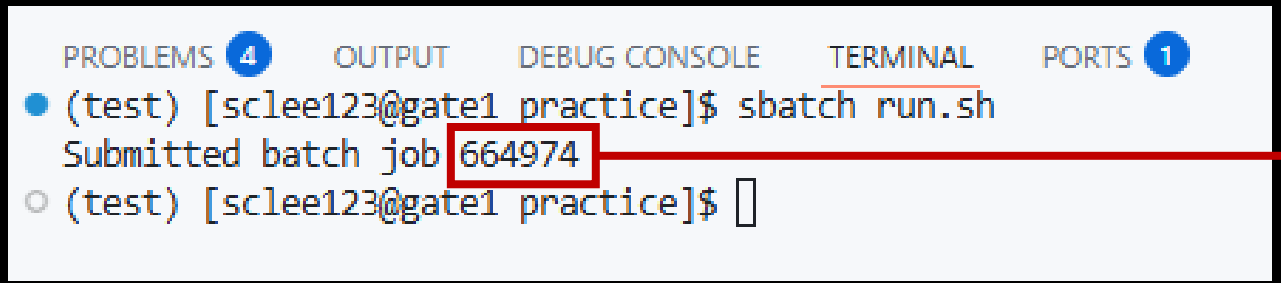
- 원하는 python 파일을 실행합니다.
- 실행하려는 파일은 .py 파일의 형태로 존재해야 합니다.
- ex) predict.py

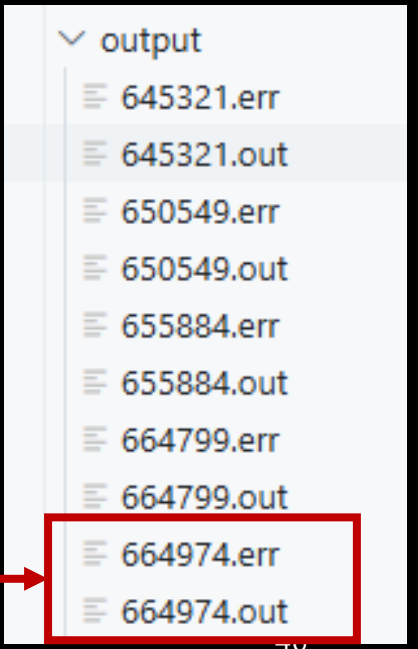
04 슈퍼컴퓨팅 환경 이해 및 Job 제출

배치 파일 실행

- Terminal에 **sbatch** 명령어를 이용하여 **배치 파일명**을 입력 및 실행하세요.
- 다음과 같이 배치 파일을 실행합니다.
  - 작업 제출 전 **ls –al** 명령어를 통해 현재 작업 디렉토리에 run.sh 파일이 존재하는지 확인합니다.
  - 존재하면, **sbatch filename.sh**를 통해 작업을 제출합니다.
  - ex) sbatch run.sh

- 이는 할당받은 노드에 작업(job)을 제출한다는 의미입니다.
  - 작업(job) 제출이 정상적으로 진행되었다면, **output 폴더** 안에 해당 작업에 대한**err/out 파일**이 각각 생성됩니다.





04 슈퍼컴퓨팅 환경 이해 및 Job 제출

배치 파일 실행

- 작업 상태 확인
  - 만일 해당 파일이 생성되지 않았다면, 해당 노드가 이미 제출한 다른 작업(job)으로 모두 할당되어 작업 대기 중일 가능성이 큽니다.
    - terminal에 qstat 명령어를 입력하여, 본인의 ID를 확인해 작업 현황을 점검하시기 바랍니다.
  - 배정이 되어 실행되고 있다면, Use 란에 R이라고 표시되어 있지만, 작업을 대기하고 있는 경우 Q라고 표시되어 있습니다.
  - 작업이 대기(Q)에 걸린 경우, 앞선 작업들이 끝나는 것을 기다려야 합니다.

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

제출 작업 확인

- qstat: 모든 제출된 작업 상태를 확인
  - 사용시 특정 사용자가 제출한 작업만을 확인하기 위해서는 –u 옵션이 필요합니다.
  - qstat –u <user_name>
  - Ex) qstat –u sclee123

  - Use – Q: 작업 대기 중

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(test) [sclee123@gate1 practice]$ qstat -u sclee123

gate1.hpc:

                                                                  Req'd  Req'd   Elap
Job id                Username Queue    Name                SessID NDS   TSK   Memory Time Use S Time
--------------------- -------- -------- ------------------- ------ ----- ----- ------ ----- - -----
664974                sclee123 gpu4     MY_JOB              --         1     1     -- 48:00 Q 00:00
```

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

제출 작업 확인

- Use – R: 작업 진행 중

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(test) [sclee123@gate1 practice]$ qstat -u sclee123

gate1.hpc:

                                                                  Req'd  Req'd   Elap
Job id               Username Queue    Name                SessID NDS   TSK   Memory Time Use S Time
-------------------- -------- -------- ------------------- ------ ----- ----- ------ ----- - -----
664974               sclee123 gpu4     MY_JOB              --         1     1     -- 48:00 R 00:00
```

- Use – C: 작업 종료

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
(test) [sclee123@gate1 practice]$ qstat -u sclee123

gate1.hpc:

                                                                  Req'd  Req'd   Elap
Job id               Username Queue    Name                SessID NDS   TSK   Memory Time Use S Time
-------------------- -------- -------- ------------------- ------ ----- ----- ------ ----- - -----
664974               sclee123 gpu4     MY_JOB              --         1     1     -- 48:00 C 00:00
```

제출 작업 취소

- 제출한 작업을 취소하기 위해서는 scancel 명령어를 사용하세요.
  - scancel <job_id> : 특정 작업만 취소합니다.
    - ex) scancel 664974

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
(test) [sclee123@gate1 practice]$ qstat -u sclee123

gate1.hpc:

                                                                  Req'd  Req'd   Elap
Job id                Username Queue    Name                SessID NDS   TSK   Memory Time Use S Time
--------------------- -------- -------- ------------------- ------ ----- ----- ------ ----- - -----
664974                sclee123 gpu4     MY_JOB              --         1     1     -- 48:00 C 00:00
```

  - scancel –u <user_name> : 자신이 제출한 모든 작업을 취소합니다.
    - ex) scancel –u sclee123
  - Use에 C가 나타났다면, 정상적으로 작업이 취소된 것입니다.
  - (Tip) Use에 C가 나타난 경우 1. 작업 수행 종료 2. (사용자 요청) 작업 취소로 인한 종료

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

기타 명령어

- pestat: 각 노드의 점유 상태 확인
- pestat –G: 각 노드의 점유 상태를 GPU 중심으로 확인
- Node State
  - idle: 현재 어떤 작업도 수행하지 않는 상태로, 해당 노드를 자유롭게 활용할 수 있습니다.
  - Mix: 일부 자원만 사용 중인 상태로, CPU 또는 GPU 자원이 남아 있습니다.
    - Case 1: CPU는 모두 점유되어 있고 GPU만 남아 있는 경우 → 추가 작업 불가
    - Case 2: GPU는 모두 점유되어 있고 CPU만 남아 있는 경우 → CPU만 사용하는 작업 가능 (GPU 사용 불가)
    - Case 3: CPU와 GPU가 모두 남아 있는 경우 → CPU 및 GPU 모두 활용 가능
    - 즉, 자신의 작업 특성에 맞는 자원이 남아 있는 노드를 선택하여 작업을 제출해야 합니다.
  - alloc: CPU와 GPU 자원이 모두 할당된 상태로, 이 노드에 작업을 제출하면 대기열(Q)에 등록됩니다.
  - resv / drain: 예약 또는 오류 상태로, 해당 노드는 작업 제출 대상에서 제외해야 합니다.

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

기타 명령어

- Num Use/CPU Tot
  - Num Use: 현재 사용 중인 CPU 코어 수
  - CPU Tot: 해당 노드의 전체 CPU 코어 수
  - Ex) 0 / 48 → 전체 48개 코어 중 사용 중인 코어가 없음 (모두 여유 상태)
  - Ex) 24 / 48 → 전체 48개 코어 중 24개 코어가 사용 중 (24 코어 이하 작업 제출 시 작업 수행 가능)
  - Ex) 48 / 48 → 전체 48개 코어가 모두 사용 중 (작업 제출 시 대기)

- Joblist (JobID User GRES/job ...)
  - JobID: 작업 고유 번호
  - User: 작업 실행 사용자
  - GRES/job ...: 사용 중인 GPU 개수
  - Ex) 500000 sclee123 gpu:rtx3090=1
  - Joblist 내 사용중인 GPU 개수가 몇 개인지 확인 필요
    - 4개 미만이면, 남은 GPU 개수만큼 GPU 사용 가능

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

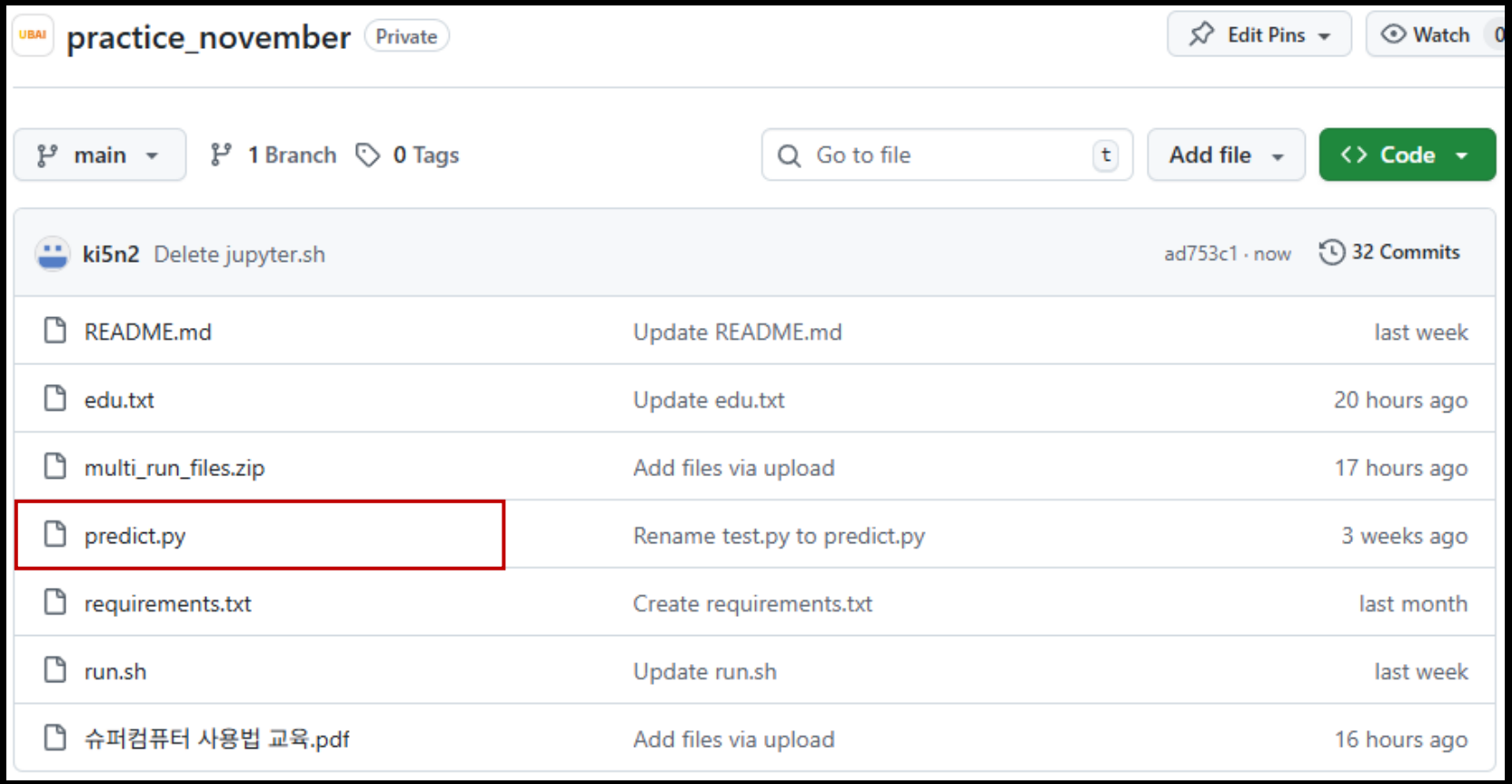
기타 명령어

- CPU / GPU 설정 안내
  - 작업을 처음 수행할 경우, CPU 코어 개수와 GPU 개수를 각각 1로 설정하여 시작
  - 이후 CPU 연산량이 많거나 GPU 연산이 집중적으로 필요한 경우, 자원 사용량을 점진적으로 늘려가며 실험을 진행
  - 매 실행 시 현재 사용 중인 CPU 코어 수 및 GPU 개수를 확인하여, 자원 사용이 적절한지 점검
  - 과도한 자원 점유를 피하고, 다른 사용자의 작업에 영향을 주지 않도록 주의
  - 자원을 과도하게 점유하거나 비효율적으로 사용하는 경우, 관리자가 모니터링 후 제재 조치가 이루어질 수 있음

04 슈퍼컴퓨팅 환경 이해 및 Job 제출

Python 프로젝트 생성

- 캘리포니아 집값 예측 테스트
  - https://github.com/uos-ubai/practice_november/blob/main/predict.py



04 슈퍼컴퓨팅 환경 이해 및 Job 제출

Python 프로젝트 생성

- 캘리포니아 집값 예측 테스트
  - pestat -G로 빈 노드를 확인한 뒤, 해당 노드·파티션에 맞게 run.sh를 수정하고 작업을 제출

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS 1
(test) [sclee123@gate1 practice]$ sbatch --reservation=edu run.sh
Submitted batch job 668516
```

practice_november / predict.py

ki5n2 Rename test.py to predict.py — 806816b · last week — History

Code | Blame | 32 lines (26 loc) · 1.01 KB — Raw

```python
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
import pandas as pd

# 1. 데이터 로딩 (sklearn 내장)
housing = fetch_california_housing()
X, y = housing.data, housing.target
print(f"데이터 로딩: {len(X):,}개 행")

# 2. 훈련/테스트 분할
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. 모델 학습
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
print("모델 학습 완료")

# 4. 예측 및 평가
predictions = model.predict(X_test)
score = r2_score(y_test, predictions)
print(f"예측 정확도: {score:.3f} ({score*100:.1f}%)")

# 5. 예측 예시
print("\n예측 예시:")
for i in range(3):
    actual = y_test[i]
    pred = predictions[i]
    print(f"실제: ${actual:.1f}만, 예측: ${pred:.1f}만 (오차: ${abs(actual-pred):.1f}만)")

print("분석 완료")
```

```
from sklearn.datasets import fetch_california_housing
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
import pandas as pd

# 1. 데이터 로딩 (sklearn 내장)
housing = fetch_california_housing()
X, y = housing.data, housing.target
print(f"데이터 로딩: {len(X):,}개 집")

# 2. 훈련/테스트 분할
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. 모델 학습
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
print("모델 학습 완료")

# 4. 예측 및 평가
predictions = model.predict(X_test)
score = r2_score(y_test, predictions)
print(f"예측 정확도: {score:.3f} ({score*100:.1f}%)")

# 5. 예측 예시
print("\n예측 예시:")
for i in range(3):
    actual = y_test[i]
    pred = predictions[i]
    print(f"실제: ${actual:.1f}만, 예측: ${pred:.1f}만 (오차: ${abs(actual-pred):.1f}만)")

print("분석 완료")
```

practice > output > 668467.out

```
start at: Mon Jun  8 15:05:00 KST 2026
node: n032
jobid: 668467
데이터 로딩: 20,640개 집
모델 학습 완료
예측 정확도: 0.805 (80.5%)

예측 예시:
실제: $0.5만, 예측: $0.5만 (오차: $0.0만)
실제: $0.5만, 예측: $0.7만 (오차: $0.3만)
실제: $5.0만, 예측: $4.9만 (오차: $0.1만)
분석 완료

```

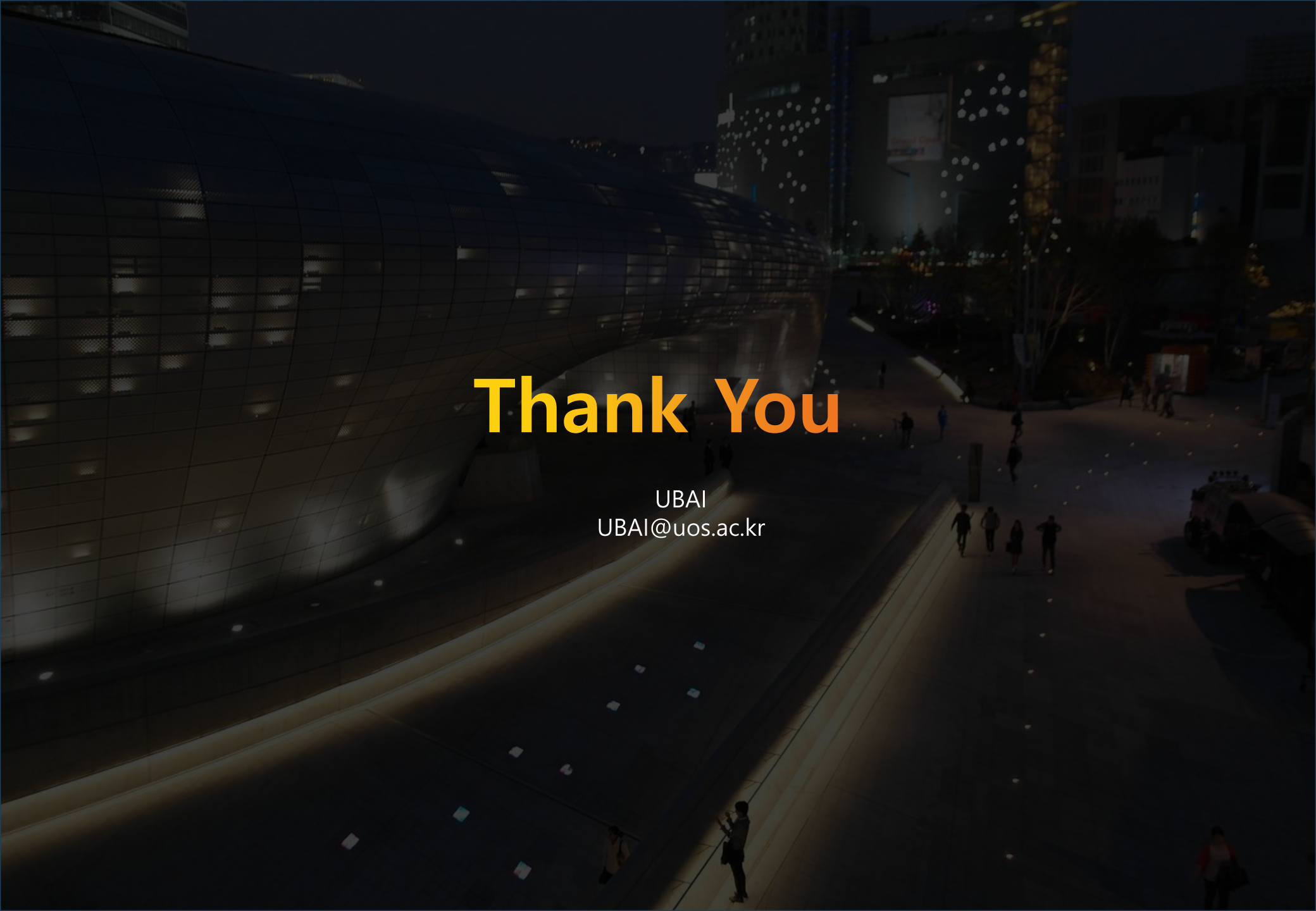
사사의 글 안내

도시과학빅데이터AI연구원의 슈퍼컴퓨터 자원을 이용하신 분들은 논문, 프로젝트, 통계 등 실적에 다음과 같이 연구원 사사를 필수적으로 적어주시기 바랍니다.

(국문) 본 논문은 서울시립대학교 도시과학빅데이터AI연구원의 슈퍼컴퓨팅 자원을 지원 받아 수행되었습니다.

(영문) The authors acknowledge the Urban Big data and AI Institute of the University of Seoul supercomputing resources (http://ubai.uos.ac.kr) made available for conducting the research reported in this paper.

UBAI



UBAI
UBAI@uos.ac.kr